Distributed Systems
Workshop 3: Naming

Semester II 2026
Sep 03, 2026
Prof. Francisco Hidrobo

Exercise 1 — Identifiers vs. Addresses

Create identifiers.py:

```
import uuid
import socket

entity_id =  uuid.uuid5(uuid.NAMESPACE_DNS, "student-a")

hostname = socket.gethostname()
ip = socket.gethostbyname(hostname)

print("Entity ID :", entity_id)
print("Hostname  :", hostname)
print("Address   :", ip)
```

Run it:

```
python3 identifiers.py
```

Now modify the program so that the address is stored separately:

```
entity = {
   "id": str(entity_id),
   "address": (ip, 5000)
}

print(entity)
```

Practical task: Student A runs the program and gives Student B only the UUID. Student B should explain why that UUID alone is insufficient to communicate with the entity.

Then modify the address:

```
entity["address"] = (ip, 6000)
```

Verify that the **identifier remains unchanged while the access address changes**.

Evidence: terminal output showing the same identifier associated with two addresses.

Exercise 2 — Locating an Entity by Broadcast

First inspect ARP/neighbour information.

Linux:

```
ip addr
ip neigh
```

Windows:

```
ipconfig
arp -a
```

Clear cache:

Linux

sudo ip neigh flush all

Windows:

arp -d *

Student A finds Student B's IP address and runs:

ping <IP-STUDENT-B>
Check the neighbour/ARP table again.

Linux:

ip neigh
Windows:

arp -a
Identify the association:

IP address → MAC address

Now reproduce the principle at application level.

Student A — entity.py

```
import socket

ENTITY_ID = "studentA-Name"
PORT = 50000

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(("", PORT))

print("Entity waiting for location requests...")

while True:
 data, addr = sock.recvfrom(1024)
 message = data.decode()

 if message == ENTITY_ID:
 response = f"{ENTITY_ID}:{socket.gethostbyname(socket.gethostname())}"
 sock.sendto(response.encode(), addr)
```

Student B — finder.py

```
import socket

ENTITY_ID = "studentA-Name"
PORT = 50000

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
sock.settimeout(3)

sock.sendto(
    ENTITY_ID.encode(),
    ("255.255.255.255", PORT)
)

try:
    data, addr = sock.recvfrom(1024)
    print("Entity found:", data.decode())
except socket.timeout:
    print("Entity not found")
```

Run entity.py on A and finder.py on B.

Student B changes:

```
ENTITY_ID = "studentB-Name"
```

and verify that no entity responds.

Evidence: successful and unsuccessful lookup.

This experiment should make the scalability limitation: broadcast requires processes to listen for location requests and does not scale beyond local-area networks.

Exercise 3 — Forwarding Pointers

Create forwarding.py:

```
locations = {
    "A": "B",
    "B": "C",
    "C": "D",
    "D": "192.168.1.50:5000"
}

def resolve(location):
    hops = 0

    while location in locations:
        print("Following:", location, "->", locations[location])
        location = locations[location]
        hops += 1

    return location, hops


address, hops = resolve("A")

print("Final address:", address)
print("Number of hops:", hops)
```

Run it.

Now implement the **shortcut/chain-reduction idea**:

```
locations["A"] = address
```
Run the lookup again.

Compare:

Before optimization: ____ hops
After optimization: ____ hops
Finally simulate a failure:

```
del locations["C"]
```
Run the original chain again and observe what happens.

Evidence: outputs before and after shortcut creation.

Exercise 4 — Implementing a Simplified Chord Ring

This is the central programming exercise.

Organize nodes into a logical ring, assign each node an *m*-bit identifier, assign entities unique *m*-bit keys, and place key k under the node with the smallest identifier ≥ k, its successor succ(k).

Part A — Chord definition
Use:

```
m = 5
```
so the identifier space is:

```
0 ... 31
```
Create:

```
nodes = [1, 4, 9, 11, 14, 18, 20, 21, 28]
```
Create chord.py:

```
M = 5
RING_SIZE = 2 ** M

nodes = [1, 4, 9, 11, 14, 18, 20, 21, 28]

def successor(key):
    for node in nodes:
        if node >= key:
            return node

    return nodes[0]

for key in [3, 8, 12, 19, 26, 30]:
    print(
        "Key:", key,
        "-> node:", successor(key)
    )
```
Verify manually where each key should be stored.

Part B — Finger tables
FT is define:

```
FTp[i] = succ(p + 2^(i-1))
```

for a node p.

Implement:

```
def finger_table(node):

    table = []

    for i in range(M):

        start = (node + 2**i) % RING_SIZE
        target = successor(start)

        table.append((i + 1, start, target))

    return table
```

Display the table for node 1:

```
for entry in finger_table(1):
    print(entry)
```

Then generate finger tables for **all nodes**.

Your output should resemble:

```
Node 1
i    start    successor
1      ...
2      ...
3      ...
4      ...
```

Part C — Lookup

Implement a lookup that uses finger-table entries instead of walking through every node.

Test with:

```
Resolve key 26 starting at node 1
```

and:

```
Resolve key 12 starting at node 28
```

Print every visited node:

```
Lookup key 26

1
→ ...
→ ...
→ successor(26)

Total hops = ...
```

Evidence: finger table for node 1 plus the complete lookup paths for keys 26 and 12.

Exercise 5 — Hierarchical Location Service

Implement a small version of HLS:

```
          ROOT
         /    \
      AMERICA  EUROPE
      /    \
   ECUADOR   USA
    /   \
 IBARRA   QUITO
```

Create hls.py:

```
tree = {
  "ROOT": {
    "AMERICA": {
      "ECUADOR": {
        "IBARRA": {},
        "QUITO": {}
      },
      "USA": {}
    },
    "EUROPE": {}
  }
}

entities = {
  "IBARRA": {
    "server01": "10.0.1.20"
  },

  "QUITO": {
    "server02": "10.0.2.30"
  }
}
```

Implement:

```
def lookup(entity, starting_domain):
  # your implementation
  pass
```

The lookup should first search locally and then move upward until it finds information indicating which subtree contains the entity.

The intended behavior: start at the local leaf; if the node knows about the entity, follow the downward pointer; otherwise move upward, with the root acting as the final upward stopping point.

Test:

server01 from IBARRA
server02 from IBARRA
server99 from IBARRA
The program must print the lookup path, for example:

IBARRA
→ ECUADOR
→ QUITO
→ server02
Then **move server01 from IBARRA to QUITO** and update the location information.

Verify that:

```
lookup("server01", "IBARRA")
```
returns its new address.

Evidence: successful lookup before and after the entity moves.

Exercise 6 — Name Spaces, Resolution, Hard Links and Soft Links

The slides model a name space as a naming graph containing directory nodes and leaf nodes; directory nodes maintain relationships to other nodes.

Create:

```
mkdir -p naming_lab/data
cd naming_lab

echo "Distributed Systems" > data/original.txt
```
Inspect the hierarchy:

```
find .
```
Create a hard link:

```
ln data/original.txt hardlink.txt
```
Create a symbolic link:

```
ln -s data/original.txt softlink.txt
```
Inspect:

```
ls -li
ls -li data/
```
Record the inode numbers.

Now execute:

```
cat hardlink.txt
cat softlink.txt
```
Rename the original:

```
mv data/original.txt data/renamed.txt
```
Try again:

```
cat hardlink.txt
cat softlink.txt
```
Record what happens.

Then delete the renamed file:

```
rm data/renamed.txt
```
Again:

```
cat hardlink.txt
cat softlink.txt
```
Explain the result: a hard link participates directly in the naming/path structure, whereas a soft link contains another name that must itself be resolved.